

Tiny Shakespeare Transformer Report

Aarush Ram Anandh

April 29, 2026

Implementation Architecture

The notebook implements a minimal Transformer language model using plain PyTorch tensors and functions rather than a full `nn.Module` class. Parameters are stored in nested Python dictionaries. The model maps a sequence of character indices to logits over the vocabulary. The flow is:

1. Read text, build a character vocabulary, and encode the text into integer tokens.
2. Embed tokens and positions, sum them, then apply a stack of Transformer blocks.
3. Apply a final layer norm and a linear output projection to produce logits.
4. Train with cross entropy loss and Adam optimizer.
5. Run inference by autoregressive sampling from the output distribution.

Causal Mask Construction

The mask prevents attention to future positions. The code creates an upper-triangular matrix and fills it with $-\infty$ above the diagonal:

$$M = \text{triu}(\mathbf{1}_{T \times T}, 1), \quad M_{ij} = \begin{cases} -\infty & i < j \\ 0 & \text{otherwise} \end{cases}$$

This is added to the attention scores before softmax so future tokens get zero probability.

How W_q , W_k , and W_v Are Built

For each attention head h , the notebook initializes learnable matrices with scaled Gaussian noise:

$$W_q^{(h)}, W_k^{(h)}, W_v^{(h)} \in \mathbb{R}^{d_h \times d_h}, \quad W \sim \mathcal{N}(0, 1)/\sqrt{d}$$

where d_h is the head dimension and d is the embedding size. The output projection $W_o \in \mathbb{R}^{d \times d}$ is initialized similarly.

Transformer Arrangement

Each block contains multi-head attention and a feedforward network, each wrapped with residual connections and layer normalization:

$$\text{Block}(x) = \text{LN}_2(\text{LN}_1(x + \text{MHA}(x)) + \text{FFN}(\text{LN}_1(x + \text{MHA}(x))))$$

The model stacks `num_layers` of these blocks, then applies a final layer norm and a linear output projection.

Function Definitions (Mathematical Form)

Below, each function is described as a mathematical mapping with its inputs and outputs.

`load_text`

$$\text{load_text}(p) = (x, \text{stoi}, \text{itos}, V)$$

Read text s from path p , build vocabulary C , and map each character to an index:

$$x_i = \text{stoi}(s_i), \quad V = |C|$$

`get_batch`

Given data $x \in \mathbb{Z}^N$, batch size B , and block size T :

$$\text{get_batch}(x, B, T) = (X, Y)$$

where

$$X_{b,t} = x_{i_b+t}, \quad Y_{b,t} = x_{i_b+t+1}$$

and i_b are random start indices.

`create_causal_mask`

$$\text{create_causal_mask}(T) = M \in \mathbb{R}^{T \times T}$$

with $M_{ij} = -\infty$ for $i < j$ and 0 otherwise.

`self_attention`

For input $x \in \mathbb{R}^{B \times T \times d_h}$ and weights W_q, W_k, W_v :

$$Q = xW_q, \quad K = xW_k, \quad V = xW_v$$

$$S = \frac{QK^\top}{\sqrt{d_h}} + M, \quad A = \text{softmax}(S)$$

$$\text{self_attention}(x) = AV$$

`multi_head_attention`

Split x into H heads and apply `self_attention` per head:

$$\text{MHA}(x) = [\text{SA}_1(x_1) \parallel \dots \parallel \text{SA}_H(x_H)] W_o$$

where $\parallel$ denotes concatenation.

`feedforward`

$$\text{FFN}(x) = \text{ReLU}(xW_1)W_2$$

`transformer_block`

$$u = x + \text{MHA}(x), \quad v = \text{LN}_1(u)$$

$$z = v + \text{FFN}(v), \quad \text{Block}(x) = \text{LN}_2(z)$$

forward

For token indices $x \in \mathbb{Z}^{B \times T}$:

$$e = E_{tok}(x) + E_{pos}(0:T-1)$$

$$h = \text{Block}_L(\dots \text{Block}_1(e) \dots)$$

$$\text{forward}(x) = hW_{out}$$

compute_loss

$$\text{compute_loss}(\ell, y) = \text{CE}(\ell, y)$$

where ℓ are logits and y are target indices.

train

Given parameters θ , the update per step is:

$$(X, Y) = \text{get_batch}(x, B, T)$$

$$\ell = \text{forward}(X), \quad L = \text{CE}(\ell, Y)$$

$$\theta \leftarrow \text{Adam}(\theta, \nabla_{\theta} L)$$

generate

For start tokens s and steps N :

$$\text{for } t = 1..N : \quad \ell = \text{forward}(x_{\text{last } T}), \quad p = \text{softmax}(\ell_{-1}), \quad x \leftarrow [x, \text{sample}(p)]$$

Function Call Flow

- Training cell: `load_text` $\rightarrow$ `init_params` $\rightarrow$ `train`.
- Inside `train`: `get_batch` $\rightarrow$ `forward` $\rightarrow$ `compute_loss` $\rightarrow$ optimizer step.
- Inference cell: `load_text` $\rightarrow$ `load_params` $\rightarrow$ `generate`.
- Extended training cell: `load_params` $\rightarrow$ `train` for more steps.

Training Procedure and Torch Operations

Training uses only core PyTorch operations: tensor indexing, `torch.randint`, `torch.stack`, `torch.triu`, `masked_fill`, `softmax`, `cross_entropy`, and `torch.optim.Adam`. Gradients are computed with `loss.backward()` and parameters are updated with `optimizer.step()`.

Input Text Embedding

Embedding does occur. Tokens are mapped to indices and passed through `nn.Embedding` to produce vectors in $\mathbb{R}^d$. A positional embedding of size `block_size` is added:

$$E(x) = E_{tok}(x) + E_{pos}(0:T-1)$$

This sum is the Transformer input.

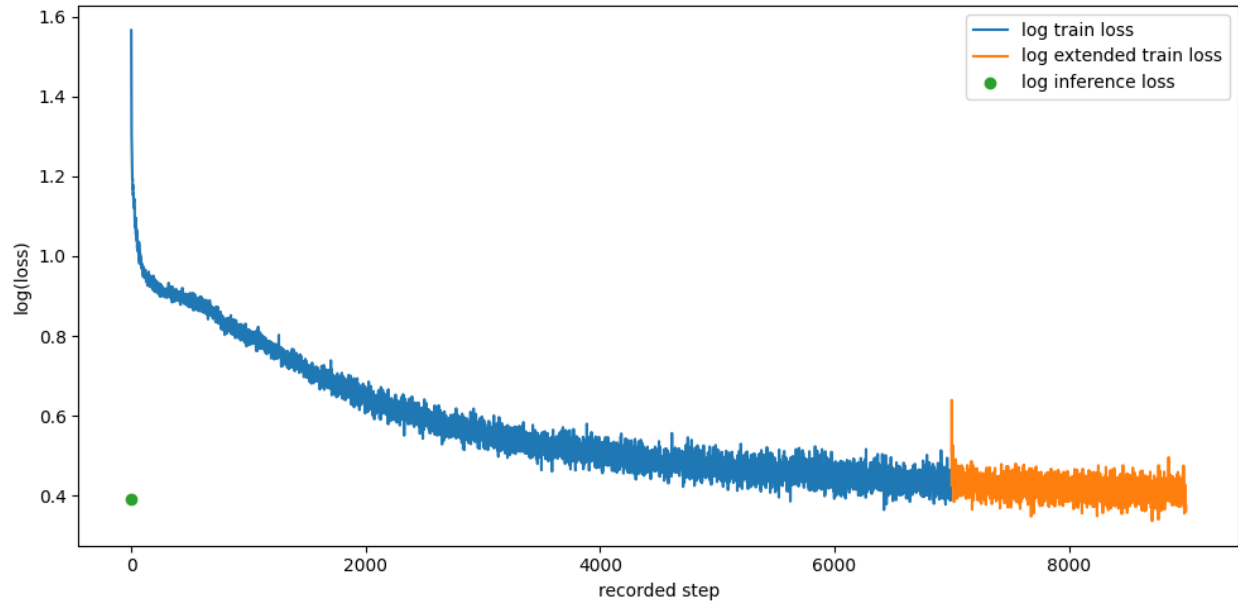


Figure 1: Training and extended training loss over recorded steps.

Results

Loss Curve

The loss decreases steadily from the initial high value and flattens as training progresses, indicating improved next-character prediction. The extended training continues from the end of the main curve and shows smaller incremental gains.

Inference Samples

Sample 1 (standard inference cell)

HASLENCE:

Is I with that him art.

BIONDELLO:

Can the therefore; but my hearty him, high nobless
 Talk the satish his bend; in jurtiful wom
 To not; for have you to thy womand's diam;
 Assarrione this donter, at now hereth,
 Lifttor: I'll are galeth art poor with our thy toook.
 That shall ever, sad do in

Sample 2 (custom prompt cell, with inference loss ≈ 1.48)

hi wowan king'd.

Methink; by be younger a dayswors cunking,
 My fathen I'll statingly.

Nill ELife:

Having to please take the forgetn, blish the gods
 The veryoners outacher'd countaly justing time,
 And coursent silf a chasqued homicted

In much a dame: I concin of your know
her rearn than breast your city, m

Inference Quality

The model shows recognizable Shakespeare-like structure (character names, line breaks, and archaic tone) but still produces many non-words and malformed phrases. This is typical for a small model trained at character level. Longer training and larger model capacity would likely improve grammatical coherence and reduce gibberish tokens.